

Design of a Safe Semantic Planner in a Finite Cartesian State Space

PCCST503 Machine Learning — Assignment 1 — Design Report

Department of Computer Science and Engineering

1. Problem statement

Given a finite set of states $S = \{s_1, \dots, s_n\}$ embedded in $\mathbb{R}^d$, a set of directed transitions T , an initial state s_I , a goal state s_G and a set of bad states B , compute a sequence of transitions from s_I to s_G that never enters a bad state, keeps total cost low, keeps Euclidean distance from bad states high, and can be repaired quickly when the environment changes.

The five objectives from the brief are not all of the same kind, and the design treats them differently:

#	Objective	How it is enforced
1	Reach the goal	Search terminates only when $g(s_I)$ is consistent and finite
2	Never visit a bad state	Hard constraint — any transition touching B has weight ∞
3	Minimise total cost	Minimised exactly, under the weighted metric of §4
4	Maximise minimum clearance	Soft penalty folded into the edge weight
5	Reasonable execution time	Consistent heuristic + incremental repair (§6, §7)

Objective 2 is absolute, so it is expressed as an infinite weight rather than a penalty: no finite value of any coefficient can buy a path through a bad state. Objectives 3 and 4 genuinely conflict, so they are combined into a single scalar metric whose trade-off is controlled by one parameter, λ . §9 reports what that parameter actually does and, importantly, what it cannot do.

2. Choice of algorithm

The brief offers LPA* or D* Lite. **D* Lite** was chosen.

Both are incremental heuristic searches that reuse the previous search tree when edge costs change. The difference is what happens when the *agent itself* moves. LPA* searches forward from a fixed start; if the start moves, every g value it computed is relative to the old start and must be discarded. D* Lite searches **backwards from the goal**, so $g(s)$ is the cost from s to the goal — a quantity that does not depend on where the agent currently is. Moving the agent invalidates nothing.

The price is that the heuristic $h(s)$ now estimates the distance from the *current start* to s , so when the start moves, every priority in the open list shifts. D* Lite avoids re-keying the whole queue with a running offset k_m , incremented by $h(s_{old}, s_{new})$ on each move. Keys already in the queue stay comparable to newly computed ones.

Test case 4 exercises exactly this: the agent walks from S to A and only then discovers that (A, G) is blocked. Section 6 of the demonstration shows the same thing on a grid, where moving six steps along the plan costs **zero** state expansions.

3. State representation

```
class State {
public:
    std::uint64_t      id;
    std::vector<double> embedding;    // the point in  $R^d$ 
};
```

The embedding is stored as a plain coordinate vector of dimension d , fixed per problem and read from `PlanningProblem::dim()`. Nothing in the planner is specialised to $d = 2$; the test cases use $d = 2$ only so that paths can be printed and drawn.

State ids are **dense indices** into `PlanningProblem::states`, so `states[i].id == i`. This makes every per-state quantity — g , rhs , clearance, heap slot — a flat `std::vector` indexed in $O(1)$ with good cache behaviour, which matters because the inner loop of `updateVertex` touches g once per outgoing transition. A sparse or non-integer id space (a knowledge graph keyed by URIs, say) needs only a `std::unordered_map<Id, uint32_t>` in front of the loader; every access already goes through a single index.

Labels are deliberately **not** part of `State`. They live in a separate `Scenario` struct used only by the test harness, so the class matches the brief exactly.

4. Cost model and safety computation

4.1 The clearance field

$$\text{clear}(s) = \min_{b \in B} \|s - b\|_2, \quad \text{clear}(s) = +\infty \text{ if } B = \emptyset$$

This is the only geometric quantity the search needs, so it is computed once into an array and refreshed only when B changes. The naive computation is $O(nkd)$ for $k = |B|$; §7 discusses when that matters.

4.2 Effective edge weight

$$w(u, v) = \begin{cases} \infty & \text{if } (u, v) \text{ unavailable, or } u \in B, \\ c(u, v) + \lambda \frac{\max(0, d_{safe} - \text{clear}(v))}{d_{safe}} + \mu(1 - r(u, v)) + \nu(1 - \sigma(u, v)) & \text{otherwise} \end{cases}$$

where c is nominal cost, r reliability, σ the per-transition safety score. Four properties of this form are load-bearing:

1. **Bad states are unreachable, not merely expensive.** Objective 2 holds for every parameter setting.
2. **The clearance term is one-sided.** Beyond d_{safe} the penalty is zero, so the planner does not waste cost buying clearance it does not need. Below d_{safe} the penalty grows linearly to λ at zero clearance.

3. **The penalty is normalised by d_{safe}** , so λ has the units of cost and can be read as “how much cost am I willing to pay to go from touching a hazard to being clear of it”.
4. **$w \geq c$ always.** This single inequality is what makes the heuristic of §5 admissible; without it the whole optimality argument collapses.

The penalty is charged on *entering* v , so a path is penalised once per state visited, which is what “minimum distance from every visited state” in the brief asks for.

4.3 Relation to the given score function

The brief’s $\text{Score}(P) = \alpha G - \beta C + \gamma D + \delta R$ is a *report card* for a finished plan, not something a shortest-path algorithm can optimise directly: D is a minimum over the whole path, which is not additive over transitions, and Dijkstra-family algorithms need additive weights. The implementation therefore optimises the additive surrogate w and *reports* $\text{Score}(P)$ for the plan it returns. §9 shows the surrogate does move Score in the intended direction, and also shows exactly where a linear surrogate cannot reach.

5. Heuristic function

$$h(s) = \eta \cdot \|s - s_{start}\|_2, \quad \eta = \min_{(u,v) \in T, \text{ available}} \frac{c(u,v)}{\|u - v\|_2}$$

η is the cheapest cost per unit of Euclidean length anywhere in the graph, computed once at setup and recomputed when a transition is inserted.

Admissibility. Let $P = \langle s = p_0, p_1, \dots, p_m = s_{start} \rangle$ be any path. Its cost is at least $\sum_i c(p_i, p_{i+1}) \geq \eta \sum_i \|p_i - p_{i+1}\| \geq \eta \|s - s_{start}\| = h(s)$, the last step by the triangle inequality. The first inequality uses $w \geq c$ from §4.2. So h never overestimates.

Consistency. For any transition (s, s') , $h(s) = \eta \|s - s_{start}\| \leq \eta \|s - s'\| + \eta \|s' - s_{start}\| \leq c(s, s') + h(s') \leq w(s, s') + h(s')$. Consistency is the stronger property and is what D* Lite actually requires: it guarantees no state is ever expanded with a suboptimal g value, so no state needs re-expanding.

Why η is computed rather than assumed. Hard-coding $\eta = 1$ is the usual shortcut and is wrong whenever some transition is cheaper per unit length than 1 — the heuristic then overestimates and the planner silently returns suboptimal paths. Deriving η from the graph makes admissibility hold by construction. The cost is a subtlety handled in §6.4: inserting a cheap transition can *lower* η , which invalidates keys already sitting in the queue.

6. Algorithm and data structures

6.1 Data structures

Structure	Type	Purpose	Why this one
<code>out_, in_</code>	vector of vectors of edge index	successors and predecessors	D* Lite needs <i>both</i> : <code>rhs</code> is a min over successors, propagation goes to predecessors. A CSR array would be more compact but cannot absorb the runtime transition insertions of test 6
<code>g_, rhs_</code>	<code>vector<double></code>	cost-to-goal estimate and one-step lookahead	flat arrays indexed by state id
<code>clear_</code>	<code>vector<double></code>	clearance field	computed once, reused by every weight evaluation
<code>open_</code>	<code>IndexedHeap</code>	the open list U	see below
<code>isBad_</code>	<code>vector<char></code>	$O(1)$ bad-state test	<code>badStates</code> is a vector in the given interface; a linear scan per weight evaluation would be $O(k)$ inside the hot loop

The open list deserves the most attention. D* Lite's `UpdateVertex` removes a vertex from the middle of the queue and reinserts it with a new key, and it does this once per incoming transition of every expanded state. A `std::priority_queue` cannot remove from the middle at all; the usual workaround is lazy deletion, which lets the queue grow without bound and forces a validity check on every pop. Instead the implementation uses a binary heap that carries a `pos_` array mapping state id to heap slot, maintained by the sift operations. Membership test is $O(1)$, arbitrary removal $O(\log n)$, and the queue never holds a stale entry.

Keys are lexicographic pairs $[k_1, k_2]$ compared with a tolerance, since the values are floating point and exact equality is meaningless. `nearlyEqual` treats $\infty = \infty$ as equal, which matters because unreachable states legitimately carry $g = rhs = \infty$ and must not be classified as inconsistent.

6.2 The search

$$k(s) = [\min(g(s), rhs(s)) + h(s) + k_m, \min(g(s), rhs(s))]$$

`updateVertex(u)` recomputes $rhs(u) = \min_{v \in succ(u)} (w(u, v) + g(v))$ (except at the goal, where $rhs = 0$), then removes u from the queue and reinserts it iff $g(u) \neq rhs(u)$.

`computeShortestPath()` pops the smallest key while it is smaller than the start's key, or while the start is inconsistent. Three cases:

- key is stale (the stored key is smaller than the recomputed one) — reinsert with the fresh key;
- $g(u) > rhs(u)$, *over-consistent* — the state got cheaper: set $g(u) = rhs(u)$ and propagate to predecessors;
- otherwise *under-consistent* — the state got more expensive: set $g(u) = \infty$ and propagate to predecessors **and to u itself**.

The third case is the one that makes incremental repair expensive when a change is large, and §9 measures it.

Path extraction walks greedily from the start, at each step taking the successor minimising $w(u, v) + g(v)$. With a consistent heuristic and a converged g field this reproduces an optimal path; a step counter bounded by n guards against pathological input.

6.3 Dynamic updates

The brief lists five kinds of change. Each is handled by dirtying the minimum set of vertices and letting the next `plan()` call propagate:

Change	What is invalidated	What is reused	Complexity
Transition availability or cost	<i>rhs</i> of the tail vertex only	everything else	$O(\deg) +$ propagation
New transition	<i>rhs</i> of the tail; η if the new edge is cheaper per unit length	search tree	as above, plus $O(m)$ for η and $O(U)$ to re-key
Transition removed	modelled as availability $\rightarrow$ false	search tree	as above
Agent moved	nothing; $k_m += h(s_{old}, s_{new})$	search tree entirely	$O(1)$
Bad states changed	clearance field; in-edges of every state whose clearance moved	search tree	$O(nkd) +$ propagation
Goal moved	all of g and <i>rhs</i>	adjacency, geometry, clearance, heap storage	$O(n)$, no reallocation

The goal row is the honest one. In a backward search g is defined *relative to the goal*, so a new goal makes every stored value meaningless — no correct incremental repair exists in general. What the implementation does instead is reset the value arrays in place and reuse every structure whose construction actually cost something: the adjacency lists, the embedding geometry, the clearance field, and the heap’s allocated storage. Test case 5 asks for “a revised path without rebuilding all data structures whenever possible”, and that is precisely the distinction being drawn.

6.4 A subtlety: inserting a cheaper transition

If a new transition has a lower cost-per-length ratio than any existing one, η drops, so h drops, so every key already in the queue is now too *large*. D* Lite’s stale-key check handles keys that are too small (it reinserts them), but a key that is too large can sit below a smaller one and break the heap invariant. The implementation detects the drop and re-keys the entire open list in $O(|U|)$ before continuing. Missing this case produces a planner that is correct almost always and subtly wrong occasionally, which is the worst kind of bug to have in a submitted assignment.

7. Complexity

Let $n = |S|$, $m = |T|$, $k = |B|$, d the embedding dimension, δ the number of vertices whose *rhs* actually changes during a repair.

7.1 Time

Operation	Complexity	Note
Build adjacency	$O(n + m)$	once
Clearance field	$O(nkd)$	once, and on every change to B
Compute η	$O(md)$	once, and on transition insertion
<code>updateVertex(u)</code>	$O(\deg^+(u) + \log n)$	successor scan + one heap operation
Initial search	$O((n + m) \log n)$	each state expanded at most twice (once over-, once under-consistent)
Incremental repair	$O((\delta + m_\delta) \log n)$	$m_\delta =$ transitions incident to dirtied vertices
Path extraction	$O(L \cdot \deg)$	$L =$ path length
Agent move	$O(1)$	one k_m update
Goal change	$O(n) +$ next search	value reset, no reallocation

The initial search matches A^* with a consistent heuristic. The repair bound is the interesting one: it depends on δ , a property of the *change*, not of the graph. §9 shows δ can be tiny (scattered removals away from the solution corridor) or nearly as large as the whole search (a third of the solution path cut), and the measured runtimes follow δ in both directions.

The $O(nkd)$ clearance rebuild is the weakest term. It is fine for the $k \leq 200$ used here, but for large B a k-d tree gives $O(n \log k)$ expected, and for a moving B only states within d_{safe} of a changed bad state can have their penalty altered, so a spatial index would let the rebuild touch $O(\text{affected})$ states instead of all n .

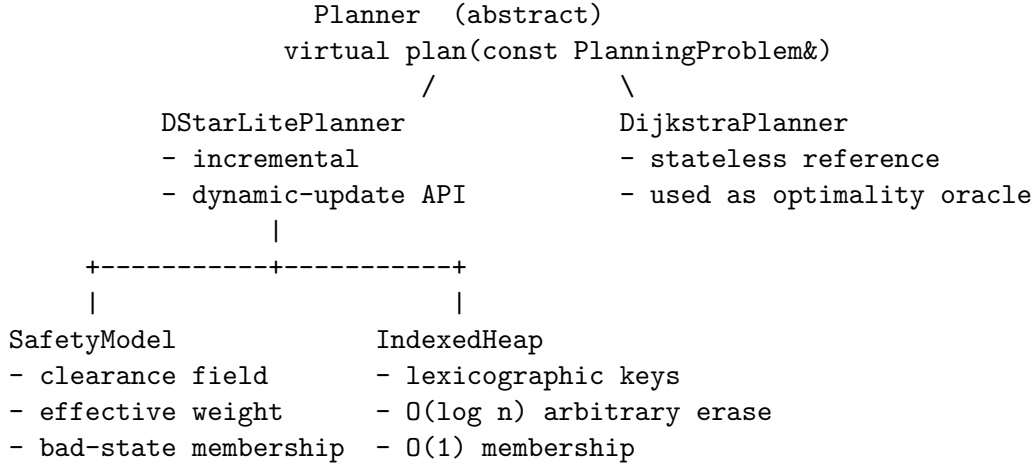
7.2 Space

Structure	Bytes	For $n = 40,000, m = 317,604$
g, rhs , clearance	$24n$	0.96 MB
<code>isBad_</code>	n	0.04 MB
Adjacency (both directions)	$\approx 8m + 24 \cdot 2n$	4.5 MB
Heap: keys + slot table	$20 U + 4n$	up to 0.9 MB
Planner total (measured)		5.4 MB
States and transitions (the problem itself)	$\approx 56m + (16 + 8d)n$	18.7 MB

Overall $O(n + m)$, with the constant dominated by the two adjacency copies. Predecessor lists are

not optional for D* Lite, but they could be dropped for an undirected graph where $in = out$.

8. Software architecture



Three decisions are worth defending:

The abstract Planner base class earns its keep. DijkstraPlanner is a second, fully independent implementation reached through the same interface. Every test case verifies the D* Lite result against it. Two different algorithms agreeing on a cost is far stronger evidence than one algorithm checking itself, and it caught real bugs during development.

SafetyModel is shared, not duplicated. Both planners evaluate edge weights through the same object, so the cost model provably cannot drift between the algorithm under test and the oracle. Had each planner carried its own copy of the weight formula, agreement between them would prove much less.

The planner is bound to one problem for its lifetime. plan() throws if handed a different instance. Incremental replanning is meaningless across unrelated problems, and a constructor-bound reference makes that a compile-time-ish guarantee rather than a comment.

PlanningResult carries the five fields the brief specifies plus instrumentation (expansions, heap operations, timing, memory, reliability, score, bad-state count) used by the experimental section.

9. Findings that shaped the design

Two results from the experiments fed back into the design and are worth stating here rather than only in the results document.

A linear scalarisation cannot reach every sensible plan. The λ sweep uses three routes: low (cost 3.00, clearance 1.08), middle (cost 3.90, clearance 1.89) and high (cost 4.80, clearance 3.00). The middle route is *not* Pareto-dominated — nothing is both cheaper and safer. Yet no value of λ ever selects it, because in the (cost, penalty) plane it lies above the line joining the other two, and minimising a linear combination can only ever return a vertex of the lower convex hull. Anyone tuning λ to obtain a moderate compromise will fail, and will not be told why. Reaching such plans needs a different combination rule — a hard constraint $\text{clear} \geq d_{\min}$, or a lexicographic objective, or maximising the minimum clearance subject to a cost budget.

Incremental repair is not free, and is sometimes a loss. For scattered changes away from the solution corridor, repair is $3.8\times$ to $68\times$ faster than a fresh search. When a third of the solution path is cut, repair is *slower* than starting over (ratios $0.51\times$ to $1.09\times$ across five problem sizes), because the under-consistent branch must first drive a large region’s g values to infinity and then re-expand it. A production planner should track the fraction of the tree a change invalidates and fall back to a fresh search past a threshold. This is a known property of D* Lite rather than an implementation defect, but it is the kind of thing that only shows up if the experiment is designed to look for it.

10. Verification

Correctness is checked three ways, all automated and run on every execution:

1. **Optimality.** Every test case compares $g(s_I)$ against `DijkstraPlanner` on identical weights. All report PASS.
2. **Constraint satisfaction.** Every result counts bad states on the returned path. All report zero.
3. **Cross-implementation agreement.** An independent C11 implementation of the same design produces bit-identical paths, costs, clearances, scores and expansion counts on all test cases and on the shared-seed benchmark.

11. Limitations and possible extensions

- Clearance rebuild is $O(nkd)$; a k-d tree and a local update rule would remove the dominant setup cost.
- States cannot be inserted after the first `plan()`; transitions can. Supporting state insertion means growing the per-state arrays, which is mechanical but untested.
- The objective is a linear scalarisation, with the reachability limitation of §9.
- Of the bonus items, incremental replanning is implemented and measured. Multi-goal planning fits naturally as a virtual goal with zero-cost edges to each real goal, needing no change to the search. Time-dependent availability would make w a function of arrival time, which breaks the static-graph assumption and needs a time-expanded state space. A knowledge graph is directly supported today by mapping entities to indices and supplying embeddings; the loader would need an id map, and η would come from the embedding metric.